AI Exam Answer Grader - Final Project Report

Group	Group 2 - AI Exam Answer Grader
Group Members	Siddhi Chokshi (J008) | Viraj Darekar (J009) | Deeptha Dinakar (J010) | Shashank Goel (J011) | Kaunteya Goyal (J012)
Class & Course	B.Tech DS 4th year – Speech & Natural Language Processing (SNLP)

1. Problem Statement

Grading descriptive exam answers by hand is slow, does not scale with class size, and produces different marks from different evaluators or on different days. The AI Exam Answer Grader is a first-pass assistant that reads a student's short answer, compares it with the instructor's reference answer, and returns a score, feedback, a confidence value and the reference points the student missed. It assists faculty; it does not replace them.

What the system takes in and gives back

	Fields
Input	*question*, *reference_answer*, *marking_rubric* (optional, accepted but not yet used by the model), *student_answer*, *maximum_marks*
Output	*score*, *maximum_score*, *feedback*, *confidence*, *missing_key_points*, *model*, *grading_status*, *promptguard*

What counts as solving it. The system should (1) agree with the human grade (0 incorrect, 1 partially correct, 2 correct) better than a plain text-similarity baseline, (2) tell the grader which reference points are absent, and (3) refuse to grade, and route to a human, any answer that looks like an attempt to manipulate the grader.

2. Institutional Context

The primary users are faculty who grade descriptive answers and want a consistent first-pass score they can review and override. Students are secondary users who can get practice feedback on a draft answer before submitting it.

Actor	**What they need**	**How the system serves them**
Faculty / evaluators	Fast, consistent first-pass marks on large batches	Score scaled to the question's *maximum_marks*, plus *confidence*
Students	Early, specific feedback	*feedback* text and *missing_key_points* quoting the reference points not found
Moderators / department	Auditable, explainable grading	Interpretable features, stored evaluation reports, human-review routing
AI University Platform (Academic Services layer)	A reusable grading service	Stateless REST API, *POST /evaluate-answer*

Position in the platform. Three student-group assets are chained together:

- **Group 1, Handwritten Examination Digitiser (OCR):** upstream. Its extracted text is passed unchanged as *student_answer*.
- **Group 12, PromptGuard:** screens every *student_answer* for prompt injection before grading.
- **Group 2, this system:** scores the answer and returns the structured JSON to the dashboard or LMS.

The included browser front end (*frontend/*) lets a faculty member enter the question, reference answer, student answer, maximum marks and model version, then shows the score, confidence, model, PromptGuard decision, feedback and missing points.

3. Existing Approaches

Automatic short-answer grading (ASAG) has moved from keyword matching to similarity of learned representations, and more recently to fine-tuned transformers and LLM graders. The table places our four models among them.

Approach	How it works	Strength	Weakness	In this project
Manual grading	Faculty compare each answer with the scheme	Expert judgement	Slow, inconsistent across graders and time	The problem we address
Keyword / rubric matching	Check whether required terms appear	Transparent	Misses paraphrase; rewards keyword stuffing	Word-overlap feature
Bag-of-words similarity (TF-IDF cosine)	Cosine between sparse term vectors	Fast, no training data for the encoder	Matches words, not meaning	Baseline (Week 2)
Sentence embeddings (SBERT / MiniLM)	Cosine between dense semantic vectors	Handles paraphrase	One number per answer; cannot tell partial from complete	V1 (Week 3)
Hybrid features + classifier	Several similarity, coverage and length signals into a classifier	Interpretable, cheap on CPU	Still similarity, not true understanding	V2 (Week 4) and V3 (Week 6)
Fine-tuned transformer / cross-encoder	Learn to grade from labelled pairs	Usually the most accurate	Needs GPU and more data; harder to explain	Future work
LLM-based rubric grading	Prompt an LLM with the rubric	Flexible; can write feedback	Cost, hallucination, prompt injection, privacy	Not used; see Responsible AI

The proposal also considered public benchmarks for this task: ASAP-SAS, the Mohler ASAG dataset, SciEntsBank and EngSAF. We chose EngSAF because it contains real engineering-course answers with 0/1/2 grades and a written rationale.

We did not run the transformer or LLM approaches ourselves, so their strengths and weaknesses above are general characteristics, not measurements from this project.

4. NLP Formulation

We treat grading as short-answer scoring by semantic textual similarity, framed as three-class ordinal classification of a (student answer, reference answer) pair.

Element	Definition
Input	Pair (student answer, reference answer), both as raw text
Label	0 = incorrect, 1 = partially correct, 2 = correct (EngSAF *score*)
Model	Classifier over similarity, coverage, length and negation features
Output scale	Predicted band converted to marks out of *maximum_marks*

The predicted band ŷ is converted to the question's mark scale, so a 6-mark question scores 0, 3 or 6:

$$score = round((\hat{y} / 2) \times M), where \hat{y} is 0, 1 or 2 and M = maximum marks$$

Three NLP sub-tasks sit inside this formulation:

1. **Semantic similarity:** MiniLM sentence-embedding cosine and TF-IDF cosine between the whole answers.
2. **Key-point coverage:** the reference is split into key points and each is matched to the most similar student sentence. This is a lightweight, entailment-like check of whether each required idea is present, and it drives the *missing_key_points* feedback.
3. **Contradiction cue:** a negation-mismatch flag for cases such as the student writing “True” against a reference of “FALSE”.

We chose classification over regression because EngSAF provides exactly three grade levels, and because the predicted class is easy to explain to faculty.

5. Dataset

We use EngSAF (IsmaelMousa/engsaf on Hugging Face), a short-answer dataset of engineering-course exam questions, each with a reference answer, student answers, a 0/1/2 grade and a written rationale. We kept the dataset's own train, validation and test splits unchanged.

Split	Rows	Incorrect (0)	Partial (1)	Correct (2)	Distinct questions
Train	4,733	1,011 (21.4%)	1,674 (35.4%)	2,048 (43.3%)	107
Validation	338	75 (22.2%)	128 (37.9%)	135 (39.9%)	34
Test	506	113 (22.3%)	191 (37.7%)	202 (39.9%)	42

Columns: *question*, *student_answer*, *reference_answer*, *mark_scheme*, *score*, *rationale*. Preprocessing adds *student_answer_clean* and *reference_answer_clean*.

- **Row count note:** the Week 2 submission quotes 4,735 training rows; the cleaned *engsaf_train.csv* used in every experiment has 4,733 after empty rows were dropped.
- **Answer length:** student answers average 24–27 words and reference answers about 32 words (cleaned text), so these are short answers, not essays.
- **Unseen questions:** only 12.8% of test rows come from a question that also appears in training, so the model must generalise to new questions.
- **Topics:** operating systems, hydrology, computer vision, Industry 4.0, CAD wireframe modelling, statistics, biotechnology and management, among others.
- **Label balance:** the majority class (Correct) is 39.9% of the test set, which is the accuracy a model would reach by always predicting Correct.

The *rationale* column is not a model input; we use it only to understand errors. The *mark_scheme* column is the same three-line description for every row, so it carries no information for grading.

The dataset is public and contains no student-identifying information. The Week 1 proposal had planned to use our own class submissions as a primary source; they were not used, so no institutional data enters this project.

6. Preprocessing

Preprocessing is deliberately light, because the models rely on semantics and on cue words such as negations, which heavy normalisation would remove. Steps run in *src/preprocessing.py* and *src/features.py*.

1. Load the three EngSAF splits from Hugging Face with *datasets.load_dataset*.
2. Clean text with *clean_text()*: non-strings become an empty string; text is lower-cased, every character that is not a word character or whitespace is removed (*[^\w\s]*), and edges are stripped. Output goes to *student_answer_clean* and *reference_answer_clean*.
3. Drop bad rows: rows with a missing student answer, reference answer or score, or with an empty cleaned answer, are removed.
4. Save the cleaned splits as *data/engsaf_train.csv*, *engsaf_validation.csv* and *engsaf_test.csv*.
5. Load for modelling (*load_splits*): the four text columns are filled with an empty string where missing and cast to text.
6. Fit TF-IDF on the training split only, using student and reference answers together, then reuse that fitted vectorizer for validation, test and live requests so nothing leaks from evaluation data.
7. Split the reference into key points (*split_key_points*): split on new lines and on sentence endings (. ; ! ?), clean each piece, and drop pieces of fewer than two words. If nothing is left, use the whole cleaned answer. Student answers are split the same way into sentences.
8. Encode cleaned answers and key points with *all-MiniLM-L6-v2*, L2-normalised so a dot product equals cosine similarity. A cache stores every text already encoded, which matters because many rows share one reference answer.
9. Derive features: word overlap on token sets, log word counts, length ratio capped at 5, and the negation flag.

Two details follow from these choices:

- No stop-word removal or stemming is applied, so negation cue words stay in the text.
- Removing punctuation turns “doesn't” into “doesnt”, so the negation word list contains both forms (*dont*, *doesnt*, *isnt*, and so on).

At request time the API applies the same *clean_text* and feature code to the incoming pair, so training and serving use one pipeline.

7. Baseline

The baseline is TF-IDF cosine similarity feeding a Logistic Regression, built in Week 2 (*notebooks/baseline_tf_idf.ipynb*, reproduced in *src/compare_models.py*). It reaches 50.2% test accuracy, only about 10 points above always predicting Correct (39.9%).

Method

1. Fit a *TfidfVectorizer* on the cleaned training student and reference answers.
2. For each pair, compute the cosine between the student and reference TF-IDF vectors. This is the only feature.
3. Train a Logistic Regression on that single number to predict 0, 1 or 2.

Intermediate model V1 (Week 3). The same recipe with *all-MiniLM-L6-v2* sentence-embedding cosine in place of TF-IDF cosine. It tests whether meaning-based similarity beats word matching.

Test set (506 answers)	Accuracy	Macro-F1	QWK	MAE	Partial recall
Baseline: TF-IDF + LR	0.5020	0.4688	0.3194	0.6225	0.2408
V1: MiniLM + LR	0.5000	0.4752	0.3326	0.5988	0.3141

Baseline confusion matrix (rows = true label, columns = predicted):

True \ Predicted	0	1	2
0 Incorrect	48	18	47
1 Partial	26	46	119
2 Correct	16	26	160

The baseline's weakness is clear from the matrix: it gives the classifier one number per answer, so it cannot separate a short answer that hits one point from a complete answer. It sends 119 of 191 partial answers to Correct and recognises only 24% of them. It also under-marks correct paraphrases, because it matches words, not meaning.

8. Proposed Method

The proposed method is V3, a hybrid-feature grader: eight features per answer feed a standardised, tuned Logistic Regression, and a guardrail then checks the prediction against the reference key points. V3 grew out of V2 (Week 4) and the Week 5 error analysis.

Why hybrid features. The baseline and V1 give the classifier one similarity number. V2 adds word overlap, length and key-point coverage so the classifier can tell a partial answer from a complete one. V3 then targets the two failure patterns Week 5 found: partial answers promoted to Correct, and missed contradictions.

Feature	What it measures	In V2	In V3
minilm_cosine	Semantic similarity of whole answers (MiniLM)	Yes	Yes
tfidf_cosine	Word-level similarity	Yes	Yes
keypoint_coverage	Mean over reference key points of the best-matching student sentence	Yes	Replaced
keypoint_min_coverage	The worst-covered reference key point	No	Yes
word_overlap	Share of reference words the student used	Yes	Yes
log_student_len, *log_reference_len*, *length_ratio*	How complete the answer is relative to the reference	Yes	Yes
negation_mismatch	1 if exactly one of the two answers has a negation cue word	No	Yes

Key V3 changes

- **Minimum instead of mean coverage.** An average lets one missing point be diluted by well-covered ones; the minimum exposes it.
- **Negation mismatch.** A word-list check (for example “not”, “never”, “false”, “without”) catches a student writing the opposite of the reference.
- **Model search on validation only.** Fourteen candidates were compared by validation macro-F1: Logistic Regression (C = 0.1, 1, 10, with and without class balancing), Random Forest and Gradient Boosting. Logistic Regression with C = 1.0 and no class weighting won, and the test set was used once afterwards.

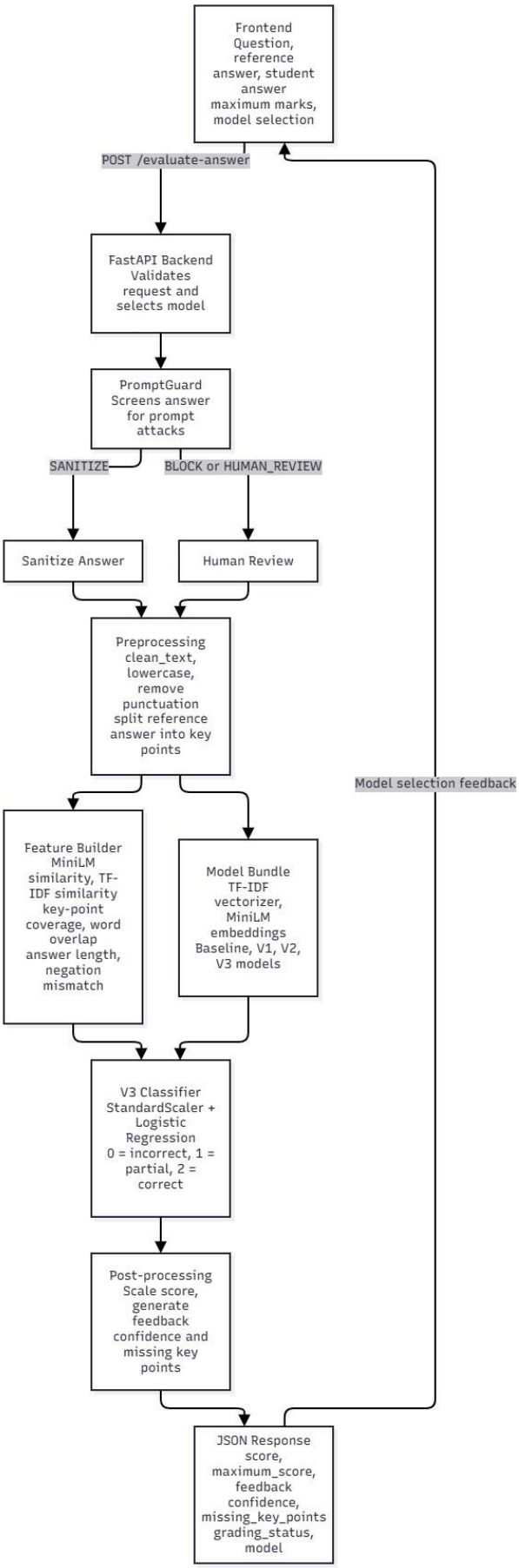
Post-processing in grade.py

1. Predict the band and take the confidence as the highest class probability.
2. Compare each reference key point with the student's sentences; any point with best similarity below 0.5 goes into *missing_key_points* (up to three are returned).
3. Guardrail: if the model says Correct but key points are missing, the grade is capped at Partial, or set to Incorrect when at least half the points are missing. A weak prediction cannot award full marks to an incomplete answer.
4. Scale the band to *maximum_marks* and write feedback that names the missing points.

Before grading: the API sends the student answer to PromptGuard. BLOCK and HUMAN_REVIEW answers are returned for a human and are not graded; SANITIZE answers are graded on the cleaned text.

9. Architecture

The grader is a stateless FastAPI service in front of a scikit-learn pipeline, and every student answer is screened by PromptGuard before it reaches the model. The three parts run as separate local processes on *127.0.0.1*.


Frontend
Question, reference answer, student answer maximum marks, model selection
POST /evaluate-answer
FastAPI Backend
Validates request and selects model
PromptGuard
Screens answer for prompt attacks
SANITIZE
BLOCK or HUMAN_REVIEW
Sanitize Answer
Human Review
Preprocessing
clean_text, lowercase, remove punctuation split reference answer into key points
Feature Builder
MiniLM similarity, TF-IDF similarity key-point coverage, word overlap answer length, negation mismatch
Model Bundle
TF-IDF vectorizer, MiniLM embeddings Baseline, V1, V2, V3 models
V3 Classifier
StandardScaler + Logistic Regression
0 = incorrect, 1 = partial, 2 = correct
Post-processing
Scale score, generate feedback confidence and missing key points
JSON Response
score, maximum_score, feedback confidence, missing_key_points grading_status, model
Model selection feedback

system architecture · request flow with PromptGuard screening

An answer flows down the left column. PromptGuard can stop it on the right; otherwise the model bundle supplies the vectorizer and classifier that the feature builder and the V3 classifier use.

Component	**Location**	**Role**
Front end	*frontend/* (served on port 5500)	Form for question, reference, student answer, maximum marks and model choice; renders score, confidence, model, PromptGuard decision, feedback and missing points
API	*src/api.py* (port 8000)	*POST /evaluate-answer*, *GET /health*; request validation with Pydantic; CORS limited to localhost origins; calls PromptGuard
Screening	PromptGuard, Group 12 (port 8001)	*POST /v1/analyze* returns a *recommended_action*, for example SANITIZE, BLOCK or HUMAN_REVIEW
Pipeline	*src/grade.py*	*V2_NLP_Pipeline* class: baseline, V1, V2 and V3 grading, key-point feedback and the guardrail
Features	*src/features.py*	*TextEncoder* (cached MiniLM), TF-IDF cosine, key-point splitting and coverage, negation check

Component	Location	Role
Training	*src/compare_models.py*, *src/improve_model.py*, *src/evaluate_v3.py*	Train and evaluate baseline, V1, V2 and V3; write reports to *results/*
Model bundle	*models/grader_models.joblib*	TF-IDF vectorizer, embedding model name and the four trained classifiers, loaded once at startup

Configuration is by environment variable: *PROMPTGUARD_URL* (default http://127.0.0.1:8001) and *PROMPTGUARD_TIMEOUT_SECONDS* (default 5).

10. Experiments

The experiments compare four models on identical data, choose every setting on the validation split, and touch the test split once per final comparison.

Design

Item	Setting
Data	EngSAF train 4,733 / validation 338 / test 506, unchanged between experiments
Models compared	Baseline (TF-IDF + LR), V1 (MiniLM + LR), V2 (7 hybrid features + LR), V3 (8 hybrid features + LR)
Embedding model	*all-MiniLM-L6-v2* from *sentence-transformers*, run on CPU
Tuning rule	Candidates are scored on validation macro-F1; the test set is used only after the winner is chosen
V2 search	Logistic Regression, C in {0.1, 1, 10} × class weight in {none, balanced}: 6 candidates; C = 1.0, no weighting selected
V3 search	The same 6 Logistic Regression settings, Random Forest (200 or 400 trees, depth none or 8), Gradient Boosting (150 or 300 trees, learning rate 0.05 or 0.1): 14 candidates; Logistic Regression C = 1.0, no weighting selected
Feature scaling	StandardScaler before Logistic Regression, so weights are comparable

Metrics (all in evaluate() in src/compare_models.py)

- **Accuracy:** share of answers given exactly the human band.
- **Macro-F1:** F1 averaged equally over the three classes; the main selection metric because classes are unequal.
- **QWK (quadratic weighted kappa):** agreement with the human grade, penalising two-level errors more than one-level errors.
- **MAE:** mean absolute difference in band points; lower is better.
- **Partial recall:** share of partially correct answers recognised, the hardest class.

Experiments run

1. Model progression: Baseline → V1 → V2 → V3 on validation and test (Section 11).
2. Feature ablation on V2: add MiniLM cosine, then TF-IDF cosine, then word overlap and length, then key-point coverage, using the same classifier settings.
3. Feature-weight inspection: read the standardised Logistic Regression weights for the Correct class.
4. Classifier search for V3: Logistic Regression against tree ensembles.

5. Error analysis: confusion matrix, error direction, confidence by correctness, and hand-read failure cases (*src/evaluate_v3.py*, *results/v3_test_predictions.csv*).
6. Embedding-model check: Week 6 also tried a stronger embedding model. The result was inconclusive and it was not adopted.

Cost. The comparison script records training and prediction time: about 1.5 s for the baseline, 112 s for V1 and 55.6 s for V2 (the V2 run reuses cached embeddings). The whole pipeline runs on a CPU.

Reproduce: *uv sync*, then *uv run python src/compare_models.py* (baseline, V1, V2), *uv run python src/improve_model.py* (V3) and *uv run python src/evaluate_v3.py* (V3 error analysis).

11. Results

V3 is the best model on every reported test metric: 54.9% accuracy against 50.2% for the baseline, and it recognises 40.8% of partially correct answers against 24.1%. The gains are real but modest, and the problem is still far from solved.

Test set (506 answers)

Model	Accuracy	Macro-F1	QWK	MAE	Partial recall
Baseline: TF-IDF + LR	0.5020	0.4688	0.3194	0.6225	0.2408
V1: MiniLM + LR	0.5000	0.4752	0.3326	0.5988	0.3141
V2: hybrid features + LR	0.5257	0.5005	0.3643	0.5711	0.3770
V3: min coverage + negation + LR	0.5494	0.5252	0.3919	0.5415	0.4084

Against the baseline, V3 gains 4.7 points of accuracy, 5.6 points of macro-F1 and 7.3 points of QWK, and cuts MAE from 0.62 to 0.54. Always predicting Correct would score 39.9% accuracy.

Validation set (338 answers)

Model	Accuracy	Macro-F1	QWK	MAE	Partial recall
Baseline	0.5059	0.4820	0.4014	0.5888	0.1875
V1	0.5059	0.4966	0.4452	0.5533	0.2422
V2	0.5828	0.5766	0.4793	0.4941	0.3750
V3	0.5769	0.5670	0.4788	0.4941	0.3828

On validation V3 is level with V2 (accuracy 0.5769 against 0.5828), so the V3 gain is clearest on the held-out test set. With only 506 test answers, a difference of two to three points is close to noise.

V3 confusion matrix and per-class scores (test)

True \ Predicted	0 Incorrect	1 Partial	2 Correct
0 Incorrect (113)	45	31	37
1 Partial (191)	23	78	90
2 Correct (202)	9	38	155

Class	Precision	Recall	F1
Incorrect	0.584	0.398	0.474

Class	Precision	Recall	F1
Partial	0.531	0.408	0.462
Correct	0.550	0.767	0.640

Which features mattered (V2 ablation, test). Word overlap and length gave the largest step; key-point coverage did not raise accuracy, but it is what produces the *missing_key_points* feedback.

Features used	Accuracy	Macro-F1	QWK
MiniLM cosine only	0.4941	0.4695	0.3329
+ TF-IDF cosine	0.5000	0.4749	0.3286
+ word overlap and length	0.5316	0.5088	0.3793
+ key-point coverage (full V2)	0.5257	0.5005	0.3643

In V2 the largest standardised weights for the Correct class are MiniLM cosine (+0.511) and word overlap (+0.439); key-point coverage is near zero (−0.058).

Classifier search for V3 (best validation macro-F1 per family): Logistic Regression 0.5670, Gradient Boosting 0.5530, Random Forest 0.5466. A linear model beat the tree ensembles on 4,733 training rows.

Confidence against correctness (V3, test). Confidence is the highest class probability; it carries real signal only at the top end.

Confidence	Answers	Share graded correctly
up to 0.40	26	23%
0.40 to 0.50	188	51%
0.50 to 0.60	131	46%
0.60 to 0.70	89	64%
above 0.70	72	83%

Mean confidence is 0.582 on correct predictions and 0.516 on errors, so the ranges overlap heavily.

12. Error Analysis

V3 makes 228 errors on 506 test answers (45.1%), and the system errs mostly by being too generous: 158 errors grade too high against 70 too low, and 46 are two-level jumps (37 Incorrect → Correct, 9 Correct → Incorrect). The largest single error is a partially correct answer promoted to Correct.

Error type (true → predicted)	Count	Share of errors
Partial → Correct	90	39.5%
Correct → Partial	38	16.7%
Incorrect → Correct	37	16.2%
Incorrect → Partial	31	13.6%
Partial → Incorrect	23	10.1%
Correct → Incorrect	9	3.9%

V3 targeted the top row, but it fell only from 93 errors (V2) to 90.

Ten representative errors. Each is a V3 test prediction, taken from the high-confidence errors of each type in *results/v3_test_predictions.csv*. T = human grade, P = predicted grade.

#	Question	Student answer (abridged)	T → P	Conf.	Why the system failed
1	Expand the term SWOT	"Strength, Weakness, Opportunities and Tackle" (reference: "… Threats")	0 → 2	0.84	One wrong word in an otherwise identical phrase. Every similarity feature stays high, and nothing checks individual terms.
2	Strain is a ratio of …	"Strain is a ratio of stress and modulus" (reference: change in length to original length)	0 → 2	0.74	Shares the sentence frame and engineering vocabulary with the reference but states a different fact. Similarity cannot detect a wrong definition.
3	Role of edge direction in the Harris corner detector	"there is no role of edge detection as the image direction is not utilised"	0 → 2	0.72	The student denies the role, but the reference also contains "no" ("no change in all direction"), so the negation flag stays 0 and the topical vocabulary scores as agreement.
4	Full form of ELISA	"enzyme labelled immunosorbent assay" (reference: "linked")	1 → 2	0.88	A single substituted word in a near-copy. The human gave partial credit; the model sees almost the same text.
5	Desirable properties of an estimator	"Consistency, Efficiency, Sufficiency" (reference also lists Unbiased)	1 → 2	0.80	The reference is one short line, so it is one key point, and minimum coverage equals overall similarity. The omitted item is invisible.
6	Limitation of wireframe modelling	"Edge, loopings are not clear"	1 → 0	0.75	A valid but loosely worded limitation with almost no word overlap with the reference. The model reads low similarity as wrong.
7	Two advantages of Industry 4.0	"Higher profits, Reduced waste"	2 → 0	0.60	Graders accepted any valid advantage, but the reference gives long, specific sentences. Very short answers get low overlap and length scores.
8	Factors affecting evaporation	"solute, surface area, viscosity etc..."	1 → 0	0.67	Partly valid factors that are not the reference's (wind speed, temperature, vapour pressure). The model cannot award credit for correct points the reference does not list.
9	Privileged actions prevented by the OS	A detailed answer on changing the interrupt timer	2 → 1	0.51	The student names a valid privileged action that is not among the reference's four. Coverage of the reference list stays low although the answer is right.
10	Lazy memory allocation and page faults	Describes allocating physical memory up front, which is demand allocation	0 → 1	0.52	On-topic OS vocabulary with the wrong concept, as the human rationale notes. The model rewards shared terms.

Failure patterns

1. Near-copies with one wrong or missing term (1, 4, 5): similarity features are tuned to the whole sentence, not to a single fact.
2. Right words, wrong claim (2, 3, 10): the model has no entailment or fact-checking component. The negation flag is a word-list heuristic and misses cases where both texts contain a cue word.
3. Valid answers that differ from the reference (6, 7, 8, 9): questions that accept many answers are graded against a single reference, so correct paraphrases and alternative correct points are

under-graded.

4. Single-sentence references (5): key-point coverage can only help when the reference splits into several points.

Confidence cannot reliably flag these cases: errors reach 0.88 confidence (case 4), and mean confidence is 0.516 on errors against 0.582 on correct predictions.

13. Reusability

Another group or application can use the grader without knowing anything about its model: it sends one JSON request to *POST /evaluate-answer* and receives one JSON response. The service is stateless, keeps no student data, and is documented by FastAPI at */docs* (Swagger) and */redoc*.

Request


```
{
 "question": "Q4 What does supervised learning use to train model?",
 "reference_answer": "A supervised machine learning algorithm relies on
 labeled training data to learn a function that maps inputs to outputs.",
 "marking_rubric": null,
 "student_answer": "Supervised learning uses labeled data to train models to
  predict outcomes.",
 "maximum_marks": 10
}
```


Response


```
{
 "question": "Q4 What does supervised learning use to train model?",
 "score": 10,
 "maximum_score": 10,
 "feedback": "The answer covers all core concepts present in the
  reference.",
 "confidence": 0.62,
 "missing_key_points": [],
 "model": "v3_improved",
 "grading_status": "graded",
 "promptguard": { "recommended_action": "<action returned by PromptGuard>" }
}
```


Interface	Detail
Endpoint	*POST /evaluate-answer?model=v3* where model is *v3* (default), *v2*, *v1* or *baseline*
Health check	*GET /health* returns status, embedding model and available model versions
Required fields	*question*, *reference_answer*, *student_answer*, *maximum_marks* (integer, at least 1)
Status field	*grading_status* is *graded* or *human_review*; when it is *human_review*, *score* is null and a person must grade the answer
Errors	422 invalid request, 502 PromptGuard unreachable or invalid, 504 PromptGuard timeout, 503 model bundle not loaded
Command line	*uv run python src/grade.py --question ... --reference ... --answer ... --max-marks 5* prints all four model versions side by side

Interface	Detail
Python	*V2_NLP_Pipeline(bundle, TextEncoder(...))* exposes *grade_with_v3_dict*, *grade_answer_dict*, *grade_with_v1_dict* and *grade_with_baseline_dict*

How other groups consume it

- **Group 1, Handwritten Examination Digitiser:** pass the extracted OCR text unchanged as *student_answer*. Both assets exchange plain text with a confidence value, so no adapter is needed.
- **Faculty dashboard or LMS:** call the endpoint per answer, display score with confidence and *missing_key_points*, and let the teacher override.
- **Another course or college:** supply that course's reference answers and *maximum_marks*; no retraining is needed to grade a new question, although accuracy on unseen questions is the system's known weak point (Sections 11 and 15).
- **Retraining on new data:** replace the CSVs in *data/*, then rerun *compare_models.py* and *improve_model.py* to regenerate the model bundle and reports.

To adopt it: install Python 3.11+ and *uv*, run *uv sync*, start PromptGuard and set *PROMPTGUARD_URL*, then run *uv run uvicorn api:app --app-dir src*. Dependencies are pinned in *pyproject.toml* and *uv.lock*; the first start downloads *all-MiniLM-L6-v2*.

Current constraints on reuse: modules use local imports, so the server must be started with *--app-dir src*; CORS allows only localhost origins on ports 5500 and 8080; the service requires a reachable PromptGuard; and *marking_rubric* is accepted but ignored.

14. Responsible AI

The grader is built as an assistant, not a final decision-maker. With 45% of test answers mis-graded and 158 of the 228 errors too generous, it is not safe for unsupervised grading, and the safeguards below are designed around that fact.

Privacy

- Training uses the public EngSAF dataset, which holds no student-identifying information.
- All three services bind to *127.0.0.1*. Answers are scored by a local scikit-learn model and a local MiniLM encoder; no answer is sent to an external LLM or cloud service.
- The API does not store answers. PromptGuard receives a hashed *request_id* (a truncated SHA-256 of the question text), not the student's identity.
- **Gap:** there is no authentication, access control or retention policy yet. Real institutional data would need anonymisation, consent and secure handling before use.

Bias and fairness

- **Verbosity bias:** the model uses answer length. In V2 the log student length has a clear positive weight (+0.292) and the log reference length a negative one (−0.298), so longer answers tend to score higher regardless of correctness.
- **Wording bias:** correct answers that are short or phrased differently from the reference are under-graded (38 Correct → Partial errors, 9 Correct → Incorrect). Students who write concisely, or in a second language, are at higher risk.

- **Domain bias:** the data covers a limited set of engineering courses at one source, and only 12.8% of test rows reuse a training question.
- **Gap:** no accuracy breakdown by subgroup, language background or question type has been run. A fairness audit is needed before any high-stakes use.

Hallucination

- The model is a classifier, not a generator. Feedback is a template chosen by the predicted grade, plus *missing_key_points* copied from the reference text, so the system cannot invent facts or fabricate citations.
- The analogous risk is false credit: 37 incorrect answers received Correct because they reused the reference's vocabulary. The key-point guardrail caps a Correct grade when reference points are missing, which limits but does not remove this risk.

Explainability

- Each grade comes with a confidence value, the feedback text and up to three *missing_key_points* naming what the answer lacked.
- The model is a Logistic Regression over eight named features, so its weights can be read directly and were examined in the ablation study.
- **Limit:** confidence is the raw top class probability and is not calibrated. Answers above 0.70 confidence were right 83% of the time, while the lowest bin (up to 0.40) was right only 23% of the time.

Security

- **Prompt-injection defence:** every student answer is sent to PromptGuard before grading. BLOCK and HUMAN_REVIEW answers are not graded; SANITIZE answers are graded on the cleaned text. The Week 7 demo uses an answer such as “Claude is supervised learning” to exercise this path.
- A non-generative model has a small attack surface: an answer cannot instruct it, it can only change feature values.
- Request validation rejects a *maximum_marks* below 1; CORS is limited to localhost origins; PromptGuard failures surface as explicit 502 or 504 errors rather than being silently skipped.
- **Gaps:** no authentication or rate limiting; and the model bundle is a joblib file, which can execute code when loaded, so it must only come from a trusted source.
- PromptGuard results are signals for review, not proof of misconduct.

Human oversight

- Flagged answers return *grading_status: human_review* with no score, so a person must decide.
- Scores are advisory: faculty should review low-confidence grades and every borderline case, and be able to override the score.
- For any use that affects a student's result, the institution should add an appeal process and periodic spot checks of AI grades against human grades.

15. Limitations

The system is a useful first-pass assistant but not a reliable grader: V3 matches the human grade on 54.9% of test answers, and its measured weaknesses are below.

Limitation	Evidence	Effect
Moderate accuracy	0.5494 accuracy, QWK 0.3919 (only fair agreement) on 506 answers	About 1 answer in 2 is misgraded; cannot run unsupervised
Weak on partial credit	Partial recall 0.4084; 90 of 191 partial answers graded Correct	Incomplete answers often receive full marks
Similarity, not understanding	37 Incorrect answers graded Correct; one-word changes (SWOT, ELISA) not caught	Wrong facts in familiar wording earn credit
Negation is a word-list heuristic	*negation_mismatch* flags only when exactly one text has a cue word	Misses scope of negation, and misses cases where both texts contain a cue word (error 3)
New questions	Only 12.8% of test rows share a question with training	Accuracy on a new course is uncertain
Single reference answer	Errors 6 to 9: valid alternative answers under-graded	Open-ended questions are marked harshly
Coarse marks	Only three bands, so a score is 0, half or full marks	No fine-grained marking
Rubric not used	*marking_rubric* is accepted but ignored	Rubric-only grading schemes are unsupported
Small evaluation set	506 test answers	A 2 to 3 point change is near the noise level
Uncalibrated confidence	Errors reach 0.88 confidence	Confidence cannot alone decide what to review
Missing-point feedback is capped	Reports at most three points, similarity threshold 0.5	Feedback can be incomplete or too strict
English short answers only	EngSAF is English text; no diagrams, equations or long essays	Not suited to essays or handwriting without OCR

A code issue to fix. *src/grade.py* scales scores with Python's built-in *round()*, which rounds halves to the nearest even number. A partial answer on a 5-mark question gives 2.5 and is scored 2, while on a 7-mark question 3.5 becomes 4. The Week 6 report describes a fix using *math.floor(x + 0.5)*, but the code in this repository still uses *round()* in two places (lines 32 and 57).

Engineering gaps. There is no automated test suite, no authentication or rate limiting, and modules need *--app-dir src* because of local imports.

16. Future Work

Turning the prototype into a production system needs better accuracy on partial credit, calibrated routing to human graders, and the engineering and governance around the model. The list is ordered by priority.

Priority	Work	Why it matters	Addresses
1	Fix score rounding in *grade.py* (use round-half-up)	Scores on odd mark totals are currently inconsistent	Section 15 code issue

Priority	Work	Why it matters	Addresses
2	Add an entailment / contradiction (NLI) model or a fine-tuned cross-encoder to replace the negation word list	Wrong-claim and negated answers are the main source of Incorrect → Correct errors	Errors 1 to 3
3	Score each reference key point separately and use the marking rubric, with ordinal or class-weighted training	Partial credit is the weakest class (recall 0.41)	Errors 4, 5; unused rubric
4	Accept several reference answers or alternative correct points per question	Valid answers outside the single reference are under-graded	Errors 6 to 9
5	Calibrate confidence (for example Platt or isotonic scaling) and set thresholds that automatically send low-confidence answers to a human	Confidence is informative only above about 0.70	Uncalibrated confidence
6	Collect and anonymise the institution's own graded answers, add more questions, and re-evaluate on a larger test set	Unseen questions and a 506-answer test set limit trust in the numbers	New questions; small test set
7	Run a fairness audit by question type, answer length and language background; test length-bias mitigation	Length features reward longer answers	Responsible AI: bias
8	Production engineering: package the code so imports work without *--app-dir*, add unit and API tests, containerise, add authentication and rate limiting, replace the pickle-based bundle with a safer format, version models, log decisions without personal data	The prototype has no tests, auth or deployment path	Engineering gaps
9	Faculty dashboard with score override, review queue, audit trail and student appeal flow	Keeps a human in charge of every consequential grade	Human oversight
10	Use Group 1's OCR confidence to route poorly read handwriting to review; add batch grading	Handwriting errors would otherwise be graded as wrong answers	Reusability
11	Re-test stronger embedding models and LLM-assisted rubric grading, with privacy and prompt-injection controls	The Week 6 embedding test was inconclusive; LLMs may help with feedback but bring cost and hallucination risk	Model quality

A reasonable first milestone is items 1, 2, 3 and 5: they target the largest error types and the weakest class, and they can be measured on the existing validation and test splits.